

Lab 01 · La ventanilla

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

45 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que la aplicación deje de terminar y se queda atendiendo, y que un método tuyo contesta desde un navegador — con el dato en la URL, en el ?, en el cuerpo, y con el código de estado que corresponde.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. La ventanilla, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 0 · La línea que lo cambia todo	3
3.2. Paso 1 · El primer endpoint	5
3.3. Paso 2 · El dato viene en la URL	7
3.4. Paso 3 · El dato viene después del ?	8
3.5. Paso 4 · Devolver un objeto, no un texto	10
3.6. Paso 5 · El dato viene en el cuerpo	11
3.7. Paso 6 · El código de estado también es la respuesta	13
4. Lo que aprendiste	15
5. Para profundizar	15
6. Antes de cerrar	15

1 Antes de empezar

1.1 Qué vas a lograr

En el Lab 00 la aplicación arrancó, imprimió y se murió en medio segundo. Hoy va a **quedarse esperando**, y un método que escribas tú va a contestar cuando alguien pregunte.

Vas a ver las cuatro formas en que un dato llega desde fuera —dentro de la ruta, detrás del ?, en el cuerpo de la petición— y vas a aprender a decidir tú el código de estado de la respuesta, en vez de aceptar el que salga. Al final vas a tener siete endpoints funcionando y vas a saber en qué se diferencian.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
El Lab 00 terminado	Lo hiciste y arrancó	No es obligatorio, pero ayuda
Estar en la carpeta del lab	<code>cd labs/lab-01-web/practica</code>	El <code>cd</code> no da error
Una forma de hacer peticiones	<code>curl --version</code>	Cualquier versión. También sirve el navegador

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de cada línea, y a veces una línea larga se parte en dos. Con Java casi nunca importa —el compilador ignora la sangría y

tu editor la vuelve a poner—, pero si una línea se parte, el editor te marcará un error justo ahí: únelas.

La red de seguridad: **el código completo está en el repositorio**, en `labs/lab-01-web/solucion/`. Si algo se pegó mal, abre ahí el mismo archivo y compara.

1.4 La puesta a punto

```
cd labs/lab-01-web/practica
```

A partir del paso 1, la aplicación **se queda corriendo**. Eso cambia cómo se trabaja:

- Para pararla, `Ctrl+C` en la terminal donde corre.
- Vas a necesitar **dos terminales**: una donde la aplicación corre, y otra donde haces las peticiones.
- Después de cada cambio en el código hay que **pararla y volver a arrancarla**.

Esta aplicación escucha en el puerto **8081**.

Si el puerto está ocupado, la aplicación te lo dirá con todas las letras y no arrancará. En el paso 1 tienes el mensaje exacto y qué hacer.

2 El caso

La DGT tiene una oficina que hasta ayer solo abría y cerraba. Hoy le pone **una ventanilla**, y alguien se sienta detrás a atender.

2.1 La ventanilla, que es la metáfora de este laboratorio

Un endpoint es una ventanilla de atención.

Cada ventanilla tiene **un número** —eso es la ruta, `/hola`, `/saludos`— y **una persona detrás** que sabe hacer una cosa: eso es tu método.

Cuando alguien llega a la ventanilla, puede traerte el dato de tres formas distintas, y las tres existen en la vida real:

- **Escrito en el cartel de la propia ventanilla**: «Ventanilla de Carolina». El dato es parte de la dirección. Eso es `@PathVariable`.
- **En las casillas de un formulario corto** que rellenas en el mostrador: «¿trato formal? sí/no». Son opcionales y llevan valor por defecto. Eso es `@RequestParam`.
- **En un formulario completo que entregas por la ranura**, con todos sus campos. Eso es `@RequestBody`.

Y lo que te devuelven no es solo un papel: es **un papel con un sello**. El sello dice si el trámite se hizo, si no existía lo que pedías, o si lo que entregaste estaba mal. Ese sello es el código de estado, y en el último paso vas a aprender a ponerlo tú.

3 Los pasos

3.1 Paso 0 · La línea que lo cambia todo

Qué vamos a hacer

Mirar el `pom.xml` de `practica/`, que ya trae una dependencia más que el del Lab 00, y arrancar para ver la diferencia.

Para entenderlo mejor

Poner una ventanilla no es tarea del conserje: hay que **encargar la ventanilla**. Eso se encarga en la lista de material — el `pom.xml`.

El problema

Una aplicación que arranca y termina no le sirve a nadie que quiera preguntarle algo. Para atender hay que quedarse escuchando en algún sitio, y eso exige un servidor web dentro del programa.

La alternativa, y por qué no

Antes esto se hacía al revés: escribías un `.war` y lo **desplegabas dentro de** un servidor — Tomcat, JBoss— que alguien había instalado y configuraba aparte. Funcionaba, pero significaba que tu aplicación no se podía arrancar sola: dependía de una máquina preparada por otra persona, y lo que probabas en tu portátil no era lo mismo que corría en producción.

Hoy el servidor **viene dentro** de la aplicación. Se arranca con `./mvnw spring-boot:run` en tu máquina exactamente igual que en un servidor, y por eso el Lab 13 podrá meterlo entero en una imagen sin instalar nada.

Se lee

Abre `practica/pom.xml`. Esta es la dependencia que no estaba en el Lab 00:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Eso es todo. Una dependencia, y la aplicación pasa de terminar a quedarse escuchando.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

```
... INFO ... o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8081 (http)
↪ with context path '/'
... INFO ... cl.dgt.web.Lab01Application      : Started Lab01Application in 0.912
↪ seconds
```

Apareció una línea que en el Lab 00 no existía: **Tomcat started on port 8081**. Y fíjate en algo más importante: **el prompt no vuelve**. La aplicación se quedó ahí.

Vas bien si...

Salí Tomcat started on port 8081 y la terminal se queda ocupada, sin devolverte el prompt. Déjala así: la vas a necesitar corriendo todo el laboratorio.

Si te atascas

1 · Web server failed to start. Port 8081 was already in use.

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

Web server failed to start. Port 8081 was already in use.

Hay otro programa escuchando en ese puerto. Casi siempre es **esta misma aplicación, arrancada en otra terminal** que se te quedó abierta: búscala y ciérrala con Ctrl+C.

Si no es eso, en macOS o Linux averigua quién lo tiene y ciérralo:

```
lsof -ti:8081 | xargs kill -9
```

En Windows:

```
netstat -ano | findstr :8081  
taskkill /F /PID <el PID de la fila que dice LISTENING>
```

2 · La terminal vuelve al prompt enseguida y no dice Tomcat started.

Estás en la carpeta del Lab 00, o en una que no tiene la dependencia web. Comprueba con pwd que estás en labs/lab-01-web/practica.

3.2 Paso 1 · El primer endpoint

Qué vamos a hacer

Crear una clase nueva con un método que conteste en /hola.

Para entenderlo mejor

Abrir la primera ventanilla y colgarle su número. El conserje —el contenedor del Lab 00— pasa, ve una ventanilla anotada como tal, y la conecta con la puerta de la calle. Tú no cableas nada.

El problema

El servidor ya está escuchando, pero no sabe qué contestar a nada. Alguien tiene que decirle **qué dirección atiende qué método**, y hacerlo sin que tengas que mantener una tabla central de rutas que se desincroniza en cuanto alguien añade una.

La alternativa, y por qué no

En los servlets de toda la vida esto se declaraba en un archivo web.xml: una lista de rutas y, al lado, el nombre de la clase que las atendía. Sacaba la información del código y la ponía en

un archivo que el compilador no revisa — un nombre de clase mal escrito no fallaba hasta que alguien pedía esa URL.

Aquí la ruta va **anotada encima del método que la atiende**. Están juntos, se leen juntos, y borrar el método borra la ruta.

Se pega

Archivo **nuevo**: practica/src/main/java/cl/dgt/web/controllers/HolaController.java. Entero:

```
package cl.dgt.web.controllers;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HolaController {

    @GetMapping("/hola")
    public String hola() {
        return "Hola, mundo.";
    }

}
```

Dos anotaciones, y cada una hace una cosa:

- **@RestController** le dice al contenedor «esto es una ventanilla». Sin ella, la clase es código muerto: nadie la mira.
- **@GetMapping("/hola")** le pone el número a la ventanilla.

Se corre

Para la aplicación con Ctrl+C, vuelve a arrancarla, y desde **otra terminal**:

```
curl localhost:8081/hola
```

Lo que vas a ver

```
Hola, mundo.
```

También puedes abrir <http://localhost:8081/hola> en el navegador. Es la misma petición.

Vas bien si...

`curl localhost:8081/hola` devuelve `Hola, mundo.` y nada más. Sin comillas, sin llaves: un texto pelado, porque el método devuelve un `String`.

Si te atascas

1 · Sale un 404 aunque el método está escrito.

```
{"timestamp":"...", "status":404, "error":"Not Found", "path":"/hola"}
```

Éste no da ningún error al arrancar, y por eso desconcierta. Casi siempre es una de dos:

- **Falta `@RestController`** encima de la clase. Sin ella el contenedor no mira la clase, y la ruta no existe para nadie. Compruébalo: es la causa número uno.
- **El archivo está en el paquete equivocado.** Spring solo escanea desde el paquete de la clase de arranque hacia abajo — aquí, `cl.dgt.web`. Si tu archivo dice otro package, no se ve. La ruta tiene que ser `src/main/java/cl/dgt/web/controllers/`.

2 · cannot find symbol

```
[ERROR] .../HolaController.java:[10,32] cannot find symbol
symbol:   class PathVariable
```

Falta una línea `import` arriba. Cada anotación que uses necesita la suya. Si tu editor tiene «organizar imports» —en IntelliJ, `Ctrl+Alt+O`—, úsalo; si no, cópiala del bloque.

3 · Cambiaste el código y sigue contestando lo de antes.

No reiniciaste. La aplicación no se entera de los cambios sola: `Ctrl+C` y `./mvnw spring-boot:run` otra vez. Es la causa más frecuente de todo el laboratorio.

3.3 Paso 2 · El dato viene en la URL

Qué vamos a hacer

Que la misma ventanilla atienda a cualquiera, con el nombre metido dentro de la dirección.

Para entenderlo mejor

El cartel de la ventanilla deja de decir «Saludos» y pasa a decir «Saludos a **Carolina**». El nombre es **parte de la dirección**, no un añadido: identifica lo que pides.

El problema

Con un método por persona no llegas a ninguna parte. Hace falta que un trozo de la ruta sea un hueco que se rellena, y que ese trozo llegue a tu método como un argumento normal.

La alternativa, y por qué no

Podrías leer la URL entera como texto y cortarla tú con `substring` y `split`. Funciona hasta el primer nombre con un acento, un espacio codificado como `%20` o una barra de más. Declarar el hueco con `{nombre}` deja que Spring haga esa conversión, y de paso deja escrito en la firma del método qué espera recibir.

Se pega

En el mismo archivo, **arriba con los demás `import`**:

```
import org.springframework.web.bind.annotation.PathVariable;
```

Y **dentro de la clase**, debajo del método anterior:

```
@GetMapping("/hola/{nombre}")
public String holaANombre(@PathVariable String nombre) {
    return "Hola, " + nombre + ".";
}
```

El nombre entre llaves y el nombre del parámetro tienen que coincidir. {nombre} arriba, String nombre abajo. Si no coinciden, Spring no sabe qué poner dónde.

Se corre

```
curl localhost:8081/hola/Carolina
```

Lo que vas a ver

```
Hola, Carolina.
```

Prueba con tu nombre. Y prueba con uno con acento: funciona, y ese es justo el trabajo que te estás ahorrando.

Vas bien si...

/hola/Carolina devuelve Hola, Carolina. y /hola/Ignacio devuelve Hola, Ignacio., sin tocar el código entre una prueba y otra.

Si te atascas

1 · 404 en /hola/Carolina, pero /hola sigue funcionando.

El método nuevo quedó **fuera de la clase** —después de la llave que la cierra— o no reiniciaste. Mira las llaves y vuelve a arrancar.

2 · Name for argument of type [java.lang.String] not specified

El nombre entre llaves no coincide con el del parámetro. Si quieres que se llamen distinto, hay que decirlo: @PathVariable("nombre") String comoSea.

3.4 Paso 3 · El dato viene después del ?

Qué vamos a hacer

Añadir una ventanilla que acepte un dato obligatorio y otro opcional con valor por defecto.

Para entenderlo mejor

Las casillas del formulario que rellenas en el mostrador. **A quién saludo** hay que ponerlo siempre; **si el trato es formal** es una casilla que, si la dejas en blanco, se entiende que no.

El problema

No todos los datos identifican lo que pides. «¿Formal o no?» no cambia *qué* recurso quieres: cambia *cómo* lo quieres. Meter eso en la ruta —/saludo/Ignacio/formal/true— produce direcciones que crecen sin control y donde el orden importa sin motivo.

La alternativa, y por qué no

Se puede leer todo de la ruta y punto. Es lo que hacen muchas API mal envejecidas, y el resultado son URLs que nadie sabe construir. La regla práctica: **lo que identifica al recurso va en la ruta; lo que lo filtra o lo modifica, va detrás del ?**.

Se pega

Arriba, con los demás import:

```
import org.springframework.web.bind.annotation.RequestParam;
```

Y dentro de la clase:

```
@GetMapping("/saludo")
public String saludo(@RequestParam String nombre,
                    @RequestParam(defaultValue = "false") boolean formal) {
    return formal ? "Buenos días, " + nombre + " : " : "Hola, " + nombre + " ";
}
```

nombre no lleva valor por defecto: es **obligatorio**. formal sí lo lleva, así que se puede omitir.

Se corre

```
curl "localhost:8081/saludo?nombre=Ignacio"
curl "localhost:8081/saludo?nombre=Ignacio&formal=true"
```

Las comillas importan: sin ellas, tu terminal se come el &.

Lo que vas a ver

```
Hola, Ignacio.
Buenos días, Ignacio.
```

Y ahora quita el parámetro obligatorio a propósito, que es la mitad del paso:

```
curl "localhost:8081/saludo"
```

```
{"timestamp":"2026-08-29T03:37:47.489Z","status":400,"error":"Bad
  ↳ Request","path":"/saludo"}
```

400, no 500. Spring distingue entre «te has equivocado tú al pedir» y «me he roto yo». Nadie escribió ese comportamiento: sale de haber declarado el parámetro como obligatorio.

Vas bien si...

Los dos curl con nombre funcionan, el segundo saluda distinto, y el curl sin nombre devuelve un **400** —no un 500 y no una página en blanco—.

Si te atascas

1 · Sale Hola, Ignacio. cuando pediste formal=true.

formal te llegó como false. Comprueba que en la URL escribiste & y no ? por segunda vez,

y que pusiste las comillas alrededor de toda la URL.

2 · La terminal se queda colgada o dice no matches found.

Te faltaron las comillas y tu shell está interpretando el ? o el &. Ponlas.

3.5 Paso 4 · Devolver un objeto, no un texto

Qué vamos a hacer

Dejar de devolver texto suelto y devolver un objeto, que Spring convierte a JSON solo.

Para entenderlo mejor

Hasta ahora entregabas **una frase escrita a mano**. Ahora entregas **un formulario impreso**, con sus campos y sus nombres. Quien lo recibe no tiene que interpretar una frase: lee el campo que le interesa.

El problema

"Hola, Carolina." es cómodo para una persona e inútil para un programa: para saber a quién va dirigido hay que trocear el texto, y el día que cambies la frase rompes a todos los que la troceaban.

La alternativa, y por qué no

Podrías construir el JSON a mano con un String y comillas escapadas. Se hace, y es una fuente inagotable de errores: una comilla dentro de un nombre y el JSON deja de ser válido.

Podrías también devolver directamente la clase interna que ya tengas. **Eso es lo que hay que evitar**, y por eso aquí hay una clase aparte: lo que sale por la ventanilla es un contrato con quien llama, y no debe cambiar solo porque cambies algo de dentro.

Se pega

Archivo **nuevo**: practica/src/main/java/cl/dgt/web/dto/SaludoDto.java. Entero:

```
package cl.dgt.web.dto;

public record SaludoDto(String mensaje, String para, boolean formal) {
}
```

Los nombres de los campos del record son las claves del JSON. Cambiarlos cambia el contrato.

En HolaController.java, **arriba con los import**:

```
import cl.dgt.web.dto.SaludoDto;
```

Necesita además la lista de nombres conocidos, que va **entre los campos de la clase**, arriba del todo:

```
private static final List<String> CONOCIDOS = List.of("mundo", "Carolina", "Ignacio");
```

Y el método, **dentro de la clase**:

```
@GetMapping("/saludos/{nombre}")
public ResponseEntity<SaludoDto> saludoDe(@PathVariable String nombre) {
    if (!CONOCIDOS.contains(nombre)) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(new SaludoDto("Hola, " + nombre + ".", nombre, false));
}
```

Hay una parte de este método que todavía no toca. El `ResponseEntity` que lo envuelve, y el `notFound()` de dentro, son el contenido del paso 6 — ahí se explican y se prueban.

Por ahora fíjate solo en una cosa: **el método devuelve un objeto**, no un texto. Lo demás llega en dos pasos.

Se corre

```
curl localhost:8081/saludos/Carolina
```

Lo que vas a ver

```
{"mensaje":"Hola, Carolina.", "para":"Carolina", "formal":false}
```

Tú devolviste un objeto Java. Lo que salió por el cable es JSON, y nadie escribió esa conversión.

Vas bien si...

La respuesta viene entre llaves, con tres claves —mensaje, para, formal— y sus valores.

Si te atascas

1 · Sale {} o faltan campos.

Devolviste una clase normal sin métodos de acceso. Con un record no pasa; si escribiste una clase, necesita getters.

2 · cannot find symbol: class SaludoDto

Falta el import en el controlador, o el archivo quedó en otro paquete. Tiene que estar en `src/main/java/cl/dgt/web/dto/`.

3.6 Paso 5 · El dato viene en el cuerpo

Qué vamos a hacer

Recibir un objeto completo en el cuerpo de la petición, con un POST.

Para entenderlo mejor

El formulario relleno que entregas por la ranura. No cabe en el cartel de la ventanilla ni en dos casillas: es un documento con varios campos, y viaja aparte de la dirección.

El problema

Los datos que se envían pueden ser largos, muchos, o privados. En la URL quedan escritos en el historial del navegador y en los registros de cualquier servidor por el que pasen. Y tienen un límite de longitud.

La alternativa, y por qué no

Se puede mandar todo por ? con un GET. Para dos campos triviales da igual; en cuanto haya datos personales, deja de dar igual — un RUT en una URL queda escrito en sitios que no controlas.

Se pega

Archivo **nuevo**: practica/src/main/java/cl/dgt/web/dto/SolicitudSaludoDto.java. Entero:

```
package cl.dgt.web.dto;

public record SolicitudSaludoDto(String nombre, boolean formal) {
}
```

Fíjate en que es distinto del de salida. El que entra y el que sale no tienen por qué ser el mismo: quien llama no decide todos los campos.

En el controlador, **arriba**:

```
import cl.dgt.web.dto.SolicitudSaludoDto;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
```

Se corre

```
curl -X POST localhost:8081/saludos \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"Ignacio","formal":true}'
```

La cabecera Content-Type no es opcional: es cómo le dices al servidor en qué idioma va el cuerpo.

Lo que vas a ver

```
{"mensaje":"Buenos días, Ignacio.,"para":"Ignacio","formal":true}
```

Vas bien si...

El POST devuelve un JSON con el saludo formal, y las claves que enviaste (nombre, formal) coinciden exactamente con los nombres de los campos del record de entrada.

Si te atascas

1 · Content type 'text/plain' not supported, o un 415.

Falta la cabecera -H 'Content-Type: application/json'.

2 · El campo llega como null o como false y tú lo mandaste.

La clave del JSON no coincide con el nombre del campo del record. Se llaman igual, y distinguen mayúsculas.

3 · Required request body is missing

Falta el `-d '...'`, o las comillas del cuerpo se rompieron. En Windows, `cmd` maneja mal las comillas simples: usa PowerShell o pon el cuerpo en un archivo con `-d @cuerpo.json`.

3.7 Paso 6 · El código de estado también es la respuesta

Qué vamos a hacer

Decidir tú el código de estado en vez de aceptar el 200 que sale por defecto.

Para entenderlo mejor

El sello del papel. Hasta ahora todo salía sellado «tramitado», incluso cuando lo que pediste no existía. Un 200 con las manos vacías es exactamente eso: un papel sellado como correcto que no dice nada.

El problema

Si todo devuelve 200, quien llama tiene que **adivinar** por el contenido si la cosa fue bien. Y las máquinas no adivinan: un cliente que reintenta, una caché, un balanceador — todos deciden mirando el código, no el cuerpo.

La alternativa, y por qué no

Podrías devolver siempre 200 con un campo `"ok": false` dentro. Es un patrón que existe y que envejece mal: obliga a cada cliente a leer el cuerpo para saber si hubo error, y desperdicia un mecanismo que ya está estandarizado y que todo el mundo entiende igual.

Se pega

Arriba:

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

El método `saludoDe` ya lo tienes escrito desde el paso 4. Vuelve a mirarlo ahora, que es cuando se entiende:

```
@GetMapping("/saludos/{nombre}")
public ResponseEntity<SaludoDto> saludoDe(@PathVariable String nombre) {
    if (!CONOCIDOS.contains(nombre)) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(new SaludoDto("Hola, " + nombre + ".", nombre, false));
}
```

`ResponseEntity<SaludoDto>` en vez de `SaludoDto` a secas es lo que te deja elegir el sello. `ResponseEntity.ok(...)` sella 200; `ResponseEntity.notFound().build()` sella 404 y no manda cuerpo — porque no hay nada que mandar.

Y el POST del paso anterior, **reemplazándolo entero**, ahora devolviendo **201**:

```
@PostMapping("/saludos")
public ResponseEntity<SaludoDto> crearSaludo(@RequestBody SolicitudSaludoDto
    solicitud) {
    String texto = solicitud.formal()
        ? "Buenos días, " + solicitud.nombre() + "."
        : "Hola, " + solicitud.nombre() + ".";
    SaludoDto saludo = new SaludoDto(texto, solicitud.nombre(), solicitud.formal());
    return ResponseEntity.status(HttpStatus.CREATED).body(saludo);
}
```

Se corre

```
curl -i localhost:8081/saludos/Carolina
curl -i localhost:8081/saludos/Pepe
curl -i -X POST localhost:8081/saludos \
    -H 'Content-Type: application/json' -d '{"nombre":"Ignacio","formal":true}'
```

El `-i` es lo que hace visible la cabecera. Sin él no ves el código de estado.

Lo que vas a ver

Para Carolina, que está en la lista: **200** y el cuerpo de siempre.

Para Pepe, que no está:

```
HTTP/1.1 404
```

404 y el cuerpo vacío. No hay error, no hay excepción: hay una respuesta que dice «eso no existe», que es la verdad.

Y para el POST:

```
HTTP/1.1 201
{"mensaje":"Buenos días, Ignacio.", "para":"Ignacio", "formal":true}
```

201, no 200. Crear algo tiene su propio código, y decirlo es gratis.

Vas bien si...

/saludos/Carolina da 200, /saludos/Pepe da 404 con el cuerpo vacío, y el POST da 201. Tres códigos distintos salidos del mismo controlador.

Si te atascas

1 · Sigue saliendo 200 en todo.

No reiniciaste, o el método que edita el estado no es el que estás llamando. Recuerda que `saludoDe` es el de `/saludos/{nombre}`, con **s**.

2 · No ves ningún código de estado.

Te falta el `-i` en el `curl`. Sin él solo se ve el cuerpo.

4 Lo que aprendiste

1 · Una dependencia convirtió el programa en un servidor.

spring-boot-starter-web trae Tomcat dentro. La aplicación dejó de terminar y se quedó escuchando, y no hubo que instalar ni configurar nada aparte.

2 · La ruta se declara donde está el método que la atiende.

@RestController marca la clase y @GetMapping marca el método. No hay una tabla central de rutas que mantener: borrar el método borra la ruta, y eso no se puede desincronizar.

3 · Un dato puede llegar de tres formas, y cada una tiene su sitio.

En la ruta lo que **identifica** lo que pides; detrás del ? lo que **filtra o modifica**; en el cuerpo lo que es **largo o privado**. Elegir mal no rompe nada hoy y te complica la vida en seis meses.

4 · El código de estado es parte de la respuesta, y lo decides tú.

Un 200 con las manos vacías obliga al que llama a adivinar. ResponseEntity te deja devolver 404 cuando algo no existe y 201 cuando algo se creó, que es información que las máquinas sí saben usar.

5 Para profundizar

Con la aplicación que ya tienes montada:

- **Añade una ventanilla que reciba dos datos en la ruta:** /hola/{saludo}/{nombre}. ¿Funciona? ¿Y si los pones al revés?
- **Haz que formal sea obligatorio** quitándole el defaultValue, y mira qué cambia cuando no lo mandas.
- **Devuelve una lista de SaludoDto** en vez de uno solo. Mira cómo sale el JSON.
- **Prueba curl -i en todos los endpoints** que escribiste hoy y anota qué código devuelve cada uno. ¿Alguno te sorprende?
- **Manda un JSON con un campo de más** en el POST. ¿Se queja? ¿Debería?

6 Antes de cerrar

La aplicación se queda corriendo. Párala con Ctrl+C en la terminal donde la arrancaste, o el puerto 8081 seguirá ocupado y el próximo arranque fallará.

```
./mvnw clean
```

Lo que te llevas:

Un endpoint es un método anotado con la ruta que atiende. El dato entra por la ruta, por el ? o por el cuerpo, y la respuesta lleva un código de estado que elige quien escribe el método.

Lo que queda pendiente, y abre el Lab 02: todo lo de hoy vive en una sola clase que se construye a sí misma. En cuanto el controlador necesite datos de verdad —de una base, de otro servicio— va a tener que pedirselos a alguien. ¿A quién, y quién decide cuál de los posibles «alguien» le toca?